

FINAL PRESENTATION · 2026.06.22

HADO Smart Court IoT

Real-time player tracking, tactical guidance,
and **17-keypoint action recognition** on Raspberry Pi 4

Jinwoo Park

Team lead · HADO Smart Court IoT

The game is real-time. The analysis is not.

HADO is an AR sport **played 3 vs 3** on a **10 × 6 m** indoor court.

Coaches review footage *after* a match — by then the moment is gone.

Our question:

Can we deliver tactical feedback **as it happens**, with just **a single camera** and a **Raspberry Pi**?

10 × 6 m

COURT · INDOOR AR ARENA

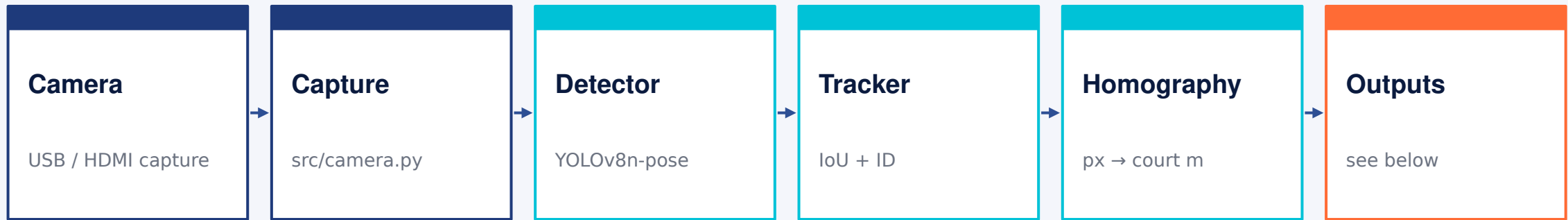
3 vs 3

PLAYERS · HEADSET-WORN AR

< 200 ms

TARGET FEEDBACK LATENCY

Single camera → Pi 4 → three-level inference stack



L1 Bird-eye tracking

Live player positions on a 10×6 m court map.

L2 Tactical engine

Six rules (R1–R6) → arrow + voice guidance.

L3 Action recognition

17 keypoints → 7 actions → next-action recommendation.

Edge hardware: \$80 of components, fully offline

Raspberry Pi 4 Model B

SPECIFICATION

Quad-core Cortex-A72 @ 1.5 GHz · 8 GB RAM
USB 3.0 × 2 · CSI camera port · Gigabit LAN

ROLE

Edge compute · Pose inference · Logging

Camera

SPECIFICATION

Arducam IMX519 16 MP CSI / GoPro · Sony via
HDMI → USB 3.0 capture board (UVC, 1080p60)

ROLE

Single-view court capture

Output

SPECIFICATION

HDMI display for on-court HUD
pyttsx3 offline TTS (Korean voice)

ROLE

Real-time visual + voice feedback

Fully offline. No cloud, no internet during operation. All inference, logging, and feedback happen on the Pi.

Where is each player, frame by frame?

1 YOLOv8n-pose @ 320 × 320

Bounding box + 17 keypoints per player, single forward pass.

2 IoU tracker

Persistent ID across frames; track buffer 60 frames.

3 Foot-point homography

Bottom-center of bbox is always on the ground plane → geometrically correct meter coords.

4 Bird-eye renderer

Live 10 × 6 m court map.

INFERENCE THROUGHPUT

Mac (development)

98 FPS

Raspberry Pi 4 (NCNN, FP16)

≥ 15 FPS · target

Why this matters

15 FPS is the minimum latency floor for tactical relevance — anything slower and the recommendation lags the action.

Six rules. One urgency arrow per player.

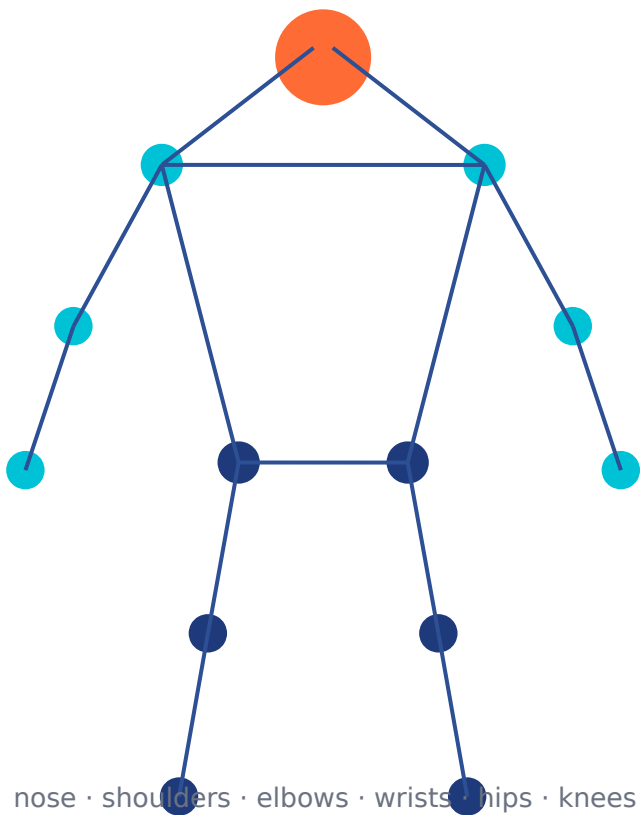
R1 Spacing Maintain ≥ 1.5 m from teammates MID	R2 Lane cover Cover open lanes toward goal MID	R3 Counter threat Block fast-approaching opponent HIGH
R4 Gap attack Exploit unguarded opponent zones MID	R5 Backline defense Keep one player near goal area HIGH	R6 Formation Maintain triangle / staggered shape MID

WHY RULES, NOT ML?

Interpretability — every arrow ships with the rule that fired it. **No labeled data** — this rule engine *is* the data collection tool for future ML work.

17 keypoints → what the player is doing — at any camera distance.

17 COCO KEYPOINTS



7 HADO-SPECIFIC ACTIONS

- ↑ **charge** wrist above shoulder, near-vertical
- ⚡ **shoot** wrist extended toward opponent side
- shield** both wrists wide, above elbows
- ← **dodge_l** shoulder midpoint left of hip midpoint
- **dodge_r** shoulder midpoint right of hip midpoint
- ↓ **crouch** shoulder–knee vertical compression
- **ready** default (no other rule fires)

CAMERA-DISTANCE INVARIANCE

scale = $\max(\text{shoulder_width}, \text{torso_height} \times 0.6, 30 \text{ px})$

All action thresholds divide by this scale, so the same logic works at 2 m or 5 m.

Not just what you ARE doing — what to do NEXT.

TRANSITION TABLE · NEXT_ACTION_RECS

current	next	reason	priority
charge	shoot	차지 완료 → 발사	1.0
charge	dodge_l	발사 포기 → 좌측 회피	0.3
shoot	dodge_r	발사 후 우측 회피	0.8
shoot	dodge_l	발사 후 좌측 회피	0.7
shield	charge	방어 해제 → 역공 차지	0.9
dodge_l	charge	회피 후 역공 차지	0.8
ready	charge	차지 시작	0.7

From WHERE to WHAT NEXT. Court position → individual action → tactical next step. One pipeline, three layers of feedback.

RECOMMENDATION LOGIC

`recommend_next_action(current, history)`

1. Look up base recommendations for current action.
2. If same action persisted ≥ 4 frames, boost transition priority $\times 1.2$.
3. Sort by priority, return top 2.

Result on screen: two recommended next actions with reasons and color-coded icons in the bottom HUD panel.

Three optimizations to hit 15 FPS on a \$35 CPU

NCNN model format

≈ 3 ×

vs. ONNX on Pi 4

ARM-optimized FP16. Native to the Pi CPU — no PyTorch, no TensorRT.

Threaded capture

0 ms

camera wait time

Background thread holds the latest frame. Inference loop never blocks on V4L2 read.

imgsz = 320

– 75 %

pixels vs. 640×640

Single biggest FPS win. 320 is still above the YOLOv8n receptive-field floor for adult bodies.

Combined effect: the full Level 1 + 3 pipeline (detector + tracker + action classifier) runs end-to-end on Pi 4 CPU only.

272 unit tests, on-court measurements, rule-level coverage

272

UNIT TESTS PASSING

15

CORE MODULES COVERED

7

ACTION-CLASS TEST SETS

≤ 10 cm

POSITION ERROR TARGET

PI 4 FIELD MEASUREMENTS · W5

metric	target	Pi 4 measured	status
NCNN inference FPS	≥ 15 fps	[TODO]	<i>pending</i>
ONNX inference FPS	≥ 10 fps	[TODO]	<i>pending</i>
RAM usage	< 1.5 GB	[TODO]	<i>pending</i>
CPU temperature (30 min)	< 70 °C	[TODO]	<i>pending</i>
TTS guidance latency	< 500 ms	[TODO]	<i>pending</i>

[TODO] slots will be populated from this week's on-court session.

See the three levels working together.



data/demo_track_a_pivot.mp4

RUN COMMANDS

Level 1 + 2

```
python -m src.main  
--level 2 --voice
```

Level 3

```
python -m src.action_demo  
--source 1 --show-kp-ids
```

Pi 4 (NCNN)

```
./run.sh action_live  
--show-kp-ids
```

What we built — and what comes next

DELIVERED

- ✓ Fully offline, single-camera tactical analysis system
- ✓ Level 1 tracking — YOLOv8n-pose + IoU tracker + homography
- ✓ Level 2 tactical engine — six interpretable rules, voice guidance
- ✓ Level 3 action recognition — 7 actions with scale-normalized classifier
- ✓ Next-action recommendation — transition table + history weighting
- ✓ Pi 4 NCNN optimization — threaded capture + imsz 320
- ✓ 272 unit tests covering all core modules

FUTURE WORK

1 Auto-tune rule thresholds

Use 635 expert-annotated movement records to fit R1–R6 weights instead of hand-coding.

2 Replace transition table with sequence model

LSTM / small transformer over 30-frame action histories.

3 Multi-camera occlusion handling

Two-camera Bayesian fusion when players overlap.

4 AR effect overlay tolerance

Robustness when HADO virtual projectiles cover the player's body.

Thank you.

Questions?

Jinwoo Park

github.com/jinwoo04/HADO_SMART_COURT_IOT

HADO Smart Court IoT · Final Presentation · 2026.06.22